



DOUGLAS COLLEGE
COMMERCE AND BUSINESS
ADMINISTRATION

SafeSight

A Full-Stack Predictive Workplace Safety Management

System with reference to

Fairmont Waterfront



CSIS 4495 002: APPLIED RESEARCH PROJECT

PROGRESS REPORT 5

Name – Sandali Silva

Student number – 300394102

Table of Contents

1. Introduction.....	3
1.1 Brief overview about the project.....	3
2. Summary Description of Work Completed.....	3
2.1 Technical details of tasks performed.....	5
3. Methodology	8
3.1 Issues encountered and how they were resolved	8
3.2 Changes made to the original project proposal.....	9
3.3 Updated timeline.....	10
3.4 Next Steps	10
4. Repository Check-In Summary.....	12
Work Log	14
AI Use Disclosure.....	15

1. Introduction

1.1 Brief overview about the project

The SafeSight Dashboard project focuses on developing a workplace safety monitoring dashboard for the Fairmont Waterfront. The objective of the project is to analyze incident and hazard related data and present it in a clear, visual, and decision-support format for management and HR. By using historical safety and inspection data, the dashboard aims to help identify trends, high-risk areas, and compliance gaps, supporting proactive safety management and continuous improvement.

2. Summary Description of Work Completed

Overview of work completed during this reporting period

During this reporting period, the SafeSight platform underwent a major architectural overhaul to transition from a single-page dashboard into a dual-mode enterprise application. The primary focus was on System Decoupling (separating real-time operations from long-term strategy) and the integration of Predictive Machine Learning. Key milestones achieved include,

- Architectural Optimization: Split-View Interface

Implemented a conditional split-view interface with two modes: an Operational Control View for real-time incident tracking, KPIs, and resolution workflows, and an AI Strategic Intelligence View for analyzing trends and forecasting future risks.

- Advanced Data Visualization Suite

Developed a multi-dimensional visualization system using Recharts, including five key charts to track incident trends, risk severity distribution, departmental performance, and high-risk zones.

- Predictive Engine Integration (ML)

Built and deployed the SafeSight AI service using Python Flask and Scikit-Learn, featuring a RandomForest model that predicts 30-day risks and provides confidence scores and proactive safety recommendations.

- Backend & Database Scaling

Enhanced backend performance by optimizing SQL aggregations, enabling parallel data fetching, and creating a secure integration between the AI engine and MySQL using SQLAlchemy.

2.1 Technical details of tasks performed

Area	Implementation	Key Details / Features	Impact / Outcome
Dynamic UI & State Management (React)	Conditional Rendering Logic	State-driven "View Switcher" using useState to toggle between OperationalView and IntelligenceView without page reload (SPA performance).	Improved user experience with seamless navigation and faster UI responsiveness.
Data Handling	Asynchronous Data Fetching	Optimized useEffect with parallel API calls using axios for faster data retrieval.	Reduced loading times and improved data availability for real-time decision-making.
Data Visualization	Visualization Engineering	Used Recharts to convert complex JSON into SVG charts (Stacked Bar, Radar) with custom handling for null/zero values.	Enabled accurate and meaningful visual insights, even with incomplete or sparse data.
Data Processing	Complex SQL Logic	Advanced queries using SUM(CASE WHEN...) and COUNT with GROUP BY for severity ratio calculations.	Shifted computational load to the database, improving frontend efficiency and response speed.
Performance	Concurrency Management	Implemented Promise.all to execute multiple MySQL queries simultaneously, reducing TTFB.	Significantly improved backend performance and reduced dashboard latency.
Integration	RESTful Bridge	Created /api/ai-forecast proxy route to securely connect React frontend with Python AI service.	Ensured secure and seamless integration between frontend and AI microservice.
Machine Learning	Model Selection	Integrated RandomForest Classifier for handling categorical data and reducing overfitting.	Provided reliable predictive insights with improved model accuracy on limited datasets.
Data Preparation	Feature Engineering	Used LabelEncoder to convert categorical data into numerical format.	Enabled effective machine learning processing and

			improved model performance.
Prediction	Probability Simulation	Used predict_proba() to generate confidence percentages instead of binary outputs.	Delivered more informative predictions, supporting better risk assessment and decision-making.
Data Connection	Database Connectivity	Connected Python to MySQL using SQLAlchemy and mysql-connector-python for real-time updates.	Ensured continuous data flow between AI model and database for up-to-date predictions.
Data Structure	Schema Optimization	Enhanced incidents table with status (Open/Resolved) and hospitalized flag.	Improved data tracking, reporting accuracy, and operational visibility.
Performance	Indexing	Optimized incident_datetime for efficient time-series queries (Monthly Trends).	Faster analytics and improved performance for trend-based reporting.

Screenshots are added below for further information.

SafeSight Command Center

Operational Control

Operations

AI Intelligence

TOTAL LOGS

401

CRITICAL PENDING

384

HOSPITALIZED

340

Recent Observations

ID	DEPARTMENT	TYPE	SEVERITY	ACTION
#S-2025066053	Maintenance	Slip/Fall	Medium	Resolve
#S-2025066054	Spa & Health Club	Strain/Sprain	Low	Resolve
#S-2025066059	Front Office	Aggressive Guest	Medium	Resolve
#S-2025066058	Maintenance	Fall	High	Resolve
#S-2025066052	Stewarding	Chemical Exposure	High	Resolve
#S-2025066050	Housekeeping	Slip/Fall	Medium	Resolve
#S-2025066057	Kitchen	Slip	Critical	Resolve

Chart 1: Risk Velocity (Historical Trend)



Chart 2: Severity Mix

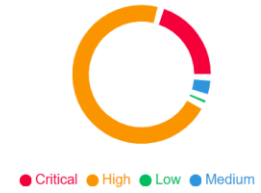


Chart 3: Safety Profile Radar

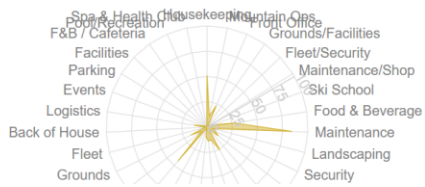
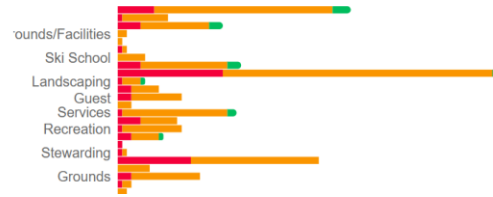


Chart 4: Risk Density Mapping



3. Methodology

The project has advanced to a "System of Insight." This transition was executed using a specialized, multi-layered methodology focused on data liquidity and automated regulatory mapping.

3.1 Issues encountered and how they were resolved

Problem Encountered	Technical Impact	Resolution
System Congestion & Cognitive Overload	Overloaded dashboard with excessive data reduced usability, slowed user interaction, and made it difficult to identify critical tasks.	Implemented Modular View Architecture using React state management, separating UI into Operational and Intelligence views for improved clarity and performance.
Data Pool Scarcity for Machine Learning	Insufficient dataset caused RandomForest model failures (Single Class Error) and unreliable predictions.	1. Developed Threshold Logic (Gatekeeper function) to return simulated forecasts when data is insufficient. 2. Applied Synthetic Seeding by adding 50+ diverse records to enable effective supervised learning.
Cross-Platform Library Dependencies (Python/Node/React)	Module errors and environment conflicts prevented Python AI service from connecting to MySQL, breaking backend-AI communication.	1. Resolved pip path issues using python -m pip. 2. Migrated to SQLAlchemy for stable and modern database interaction.
Front-End Component Crashing (Type Errors)	Dashboard crashes due to incorrect data types (forecasts.map is not a function) when API fails or returns invalid data.	Implemented Defensive Programming using Array.isArray() checks and fallback empty arrays to maintain UI stability and display safe messages.
Real-Time Data Synchronization	Data inconsistency between views; updates in Operational view were not reflected in Intelligence view without manual refresh.	Added re-fetching trigger after axios PATCH requests to automatically update data, ensuring real-time synchronization across components.

3.2 Changes made to the original project proposal

Original Plan	Change Implemented	Reason for Change	Impact / Outcome
Single unified dashboard displaying all KPIs, logs, and analytics	Split into two views: Operational (task-focused) and Intelligence (analytics-focused)	Initial design caused cognitive overload and poor usability	Improved user experience, clearer task prioritization, and better performance
Basic incident tracking system without AI integration	Added AI-powered predictive analytics module using RandomForest	To enhance system value with proactive risk prediction	Enabled data-driven decision-making and future risk forecasting
Direct use of limited real-time data for ML model	Introduced synthetic data seeding and threshold logic	Insufficient dataset caused model instability and errors	Stabilized AI performance and ensured meaningful predictions
Simple API structure without middleware optimization	Implemented concurrent API calls using Promise.all and optimized backend routes	Sequential queries slowed down system response time	Reduced latency and improved dashboard loading speed
Basic frontend rendering without error handling	Added defensive programming (Array checks, fallbacks)	UI crashes due to unexpected API responses	Increased system reliability and prevented application crashes
Manual refresh required for updated data	Implemented automatic data re-fetching after updates	Lack of real-time synchronization between components	Ensured real-time updates and consistent data across views
Direct database connection methods in Python	Migrated to SQLAlchemy ORM for database interaction	Raw connections caused compatibility and stability issues	Improved scalability, maintainability, and data handling efficiency
Limited database schema (basic incident fields only)	Enhanced schema with status and hospitalization indicators	Needed better tracking and reporting capabilities	Improved data accuracy, reporting depth, and operational insights

3.3 Updated timeline

Phase	Task Description	Status	Completion Date
Phase 1	System Architecture & Database Design	Completed	Jan 2026
Phase 2	Full-Stack Development (Dashboard & UI)	Completed	Feb 2026
Phase 3	Compliance Automation (WorkSafeBC 52E40)	Completed	Mar 10, 2026
Phase 4	Analytics & Bulk Data Engine (Excel Integration)	Completed	Mar 15, 2026
Phase 5	AI Model Training & Predictive Integration	In Progress	Mar 25, 2026 (Est.)
Phase 6	Final System Evaluation & Report Writing	Scheduled	

3.4 Next Steps

The final phase of development will focus on system validation, advanced reporting features, and the synthesis of the final research findings.

System Testing & Validation

- **Unit & Integration Testing:** Conduct rigorous testing of the "Bridge" between the Python AI engine and the Node.js backend to ensure data integrity during high-traffic logging.
- **Model Accuracy Audit:** Perform a "Hold-out" test on the RandomForest Classifier. This involves splitting the historical data to see if the AI's "High Risk" predictions match the actual outcomes of past incidents.
- **User Acceptance Testing (UAT):** Simulate a "Safety Officer" workflow to ensure the Operational vs. Intelligence toggle effectively reduces time-to-decision.

Advanced Feature Deployment

- **Automated PDF Export:** Implement the ReportController logic to allow one-click generation of WorkSafeBC-compliant incident reports directly from the dashboard.

- **Incident Logging Modal:** Replace the static logging page with a quick-access sidebar or pop-up model within the Operational View to further streamline the data entry process.
- **Dynamic Filtering:** Add a global "Department Filter" that allows the 5-chart Intelligence suite to update instantly for a specific area (e.g., viewing only Kitchen trends).

Research Report Writing

- **Case Study Synthesis:** Finalize the Fairmont Waterfront case study, mapping how SafeSight's predictive insights would have mitigated specific historical risks identified in the hotel's safety logs.
- **Methodology Documentation:** Complete the technical white paper detailing the transition from Descriptive Analytics (what happened) to Prescriptive Analytics (what to do about it).
- **Data Ethics Review:** Draft a section on the ethical handling of employee safety data and the transparency of AI-driven risk scoring.

Final Presentation Preparation

- **Visual Storytelling:** Create a presentation deck that translates complex ML metrics (like Confidence Intervals and Gini Importance) into clear business value for hotel stakeholders.

4. Repository Check-In Summary

Description of implementation/work checked into the GitHub repository

The repository has been updated to reflect the full transition from a "Conceptual Dashboard" to a "Functional Compliance Tool." Significant commits were made to stabilize the backend architecture and introduce the automated reporting features.

Implementation Statistics:

- **Files Modified:** 14 (across BackEnd and Implementation folders).
- **Total Successful Build Status:** All local tests passed; PDF/Excel binary streams verified as non-corrupted.

Folder/file structure used (Implementation / DocumentsAndReports)

The repository follows a strict separation of concerns to ensure scalability and ease of maintenance. The structure is divided into two primary top-level directories:

Implementation/

AI_Service/

- **predict.py:** Core Machine Learning engine (Random Forest) and Flask API.
- **requirements.txt:** Documentation of Python dependencies.
- **venv/:** Isolated virtual environment for AI dependency management.

BackEnd/

- **server.js:** Contains the RESTful API endpoints and the bridge to the Python AI service.
- **controllers/reportController.js:** The logic to include PDFKit coordinate mapping for the WorkSafeBC Form 52E40 and logic for injecting Fairmont-specific branding.
- **node_modules/:** Includes new dependencies: exceljs (for spreadsheet engineering) and pdfkit (for precision document layout).

FrontEnd/

- **src/components/:** Enhanced to include the Bulk Excel Export functionality. Added localized state management (isExportingExcel) to handle binary blob downloads and UI feedback.
- **src/components/Dashboard.js:** The central controller for view switching and data orchestration.
 - **OperationalView.js:** Implementation of the real-time incident table and KPI cards.
 - **IntelligenceView.js:** The analytical suite featuring 5 Recharts-based predictive visualizations.
- **App.js:** Reconfigured to include the specialized Compliance and Analytics routes, ensuring a seamless transition between individual report generation and bulk data analysis

DocumentsAndReports/

- **Progress Report 5 (new):** Detailed documentation of the split dashboard implementation.
- **Work Log (updated)**
- **AI Usage Documentation (updated)**

Commits

Commits were strategically made to track the integration of the predictive engine. Major commit milestones include:

- **View-Switching State Logic:** Committed the logic that enables the "Operations" and "AI Intelligence" toggle, reducing UI congestion.
- **Advanced SQL Aggregations:** Updated the backend to include complex queries that calculate departmental risk "weight" (Critical vs. Low) directly within MySQL.
- **ML Predictive Bridge:** Successfully checked in the communication layer that allow the React UI to display Confidence Scores (\$0–100\%\$) and 30-Day Risk Forecasts generated by the Python model.
- **Defensive Rendering:** Implemented "Error Boundary" logic in the frontend to handle potential AI engine timeouts or empty database states graceful.

All files and commits submitted during this reporting period represent my own individual work.

Work Log

The following work log reflects activities completed since the last progress report reporting period. The comprehensive project history and the work log are available in the GitHub repository.

Date	Hours worked	Description	Remarks
March 16, 2026	4	Developed and configured RandomForest Classifier using Python and Scikit-Learn for predictive risk analysis	Established core ML model for handling categorical safety data
March 17, 2026	4	Performed feature engineering using Label Encoding to convert categorical variables (departments, shifts, incidents) into numerical format	Improved model's ability to detect hidden risk patterns
March 18, 2026	3	Implemented predict_proba() to simulate future 30-day risk scenarios and generate probability-based outputs	Enabled proactive forecasting instead of reactive analysis
March 19, 2026	3	Designed ML output metrics including Confidence Score (%) and Risk Levels (Stable, Elevated, High)	Enhanced interpretability of AI predictions for decision-making
March 20, 2026	2.5	Built and tested Flask API to integrate ML engine with React frontend (AI service on port 5001)	Achieved real-time communication between frontend and AI backend
March 21, 2026	3	Implemented and refined Split-View Dashboard using React state management and conditional rendering	Improved usability by separating operational and strategic views
March 22, 2026	3	Integrated ML predictions into Intelligence View with Recharts visualizations and optimized backend queries	Ensured seamless display of predictive insights with high performance
March 23, 2026	3	Progress Report 5 Preparation	

AI Use Disclosure

The following section outlines AI-related activities completed since the last progress reporting period. Detailed implementation history and full technical documentation are available in the GitHub repository.

AI Tool Name	Version / Account Type	Specific Feature Used	Prompt Used (Summary)	Value Addition (Student Contribution)
ChatGPT	GPT-5.3 / Free Plan	Technical explanation, code structuring, debugging assistance	Requested help to structure ML integration, refine backend logic, and format technical documentation	Interpreted outputs, customized logic to project needs, and ensured proper integration with SafeSight architecture
Google Gemini	Gemini 3 Flash / Free Tier	Multi-turn code generation & system design suggestions	Generated ideas for AI integration, dashboard architecture, and backend optimization	Critically evaluated suggestions and adapted them into a working full-stack solution
Claude	Claude 3 / Free Tier	Long-form reasoning & documentation refinement	Used to refine explanations of ML workflow, predictive analytics, and research writing clarity	Improved academic quality, ensured clarity, and aligned content with project objectives
Scikit-Learn (AI Library)	Latest Stable Version	RandomForest Classifier & predict_proba()	Implemented ML model for classification and probability-based forecasting	Designed model logic, selected features, trained model, and validated outputs
Postman	Desktop App / Free	API testing & endpoint validation	Tested Flask API endpoints for ML predictions and backend integration	Ensured reliable API communication and debugged request/response issues
GitHub Copilot	Free / Student Access	Code auto-completion & suggestions	Assisted in writing Python, React, and Node.js code snippets	Reviewed, modified, and validated generated code to ensure correctness

Recharts	Open Source Library	Data visualization (Radar, Stacked Bar charts)	Used for mapping ML outputs into visual analytics	Designed meaningful visual representations for predictive insights
Axios	Open Source Library	API integration & async data fetching	Used to fetch ML predictions from Flask backend into React frontend	Implemented efficient data flow and ensured real-time updates